

Курсовая работа (проект) по системному программированию (2025-2026 уч.г.)

Варианты:

1. Используемый индекс: B-tree
2. Используемый индекс: B+-tree
3. Используемый индекс: B*-tree
4. Используемый индекс: B*+-tree

0. На языке программирования C++ (стандарт C++14 и выше) реализуйте систему управления базами данных (СУБД), поддерживающую работу с целочисленными (int) и строковыми (string) типами данных.

Система хранения данных должна иметь двухуровневую иерархическую структуру:

- **Уровень систем:** содержит коллекцию баз данных.
- **Уровень базы данных:** содержит коллекцию таблиц.
- **Уровень таблицы:** содержит типизированные данные (записи).

Взаимодействие с СУБД осуществляется посредством SQL-подобного языка запросов (синтаксис описан ниже). Приложение должно поддерживать два режима работы, переключаемых через аргументы командной строки:

1. **Интерактивный режим** (запуск без аргументов ./prog): команды вводятся пользователем в терминал, результат выводится немедленно.
2. **Пакетный режим** (запуск вида ./prog script.txt): выполнение последовательности команд, считанных из указанного файла.

Требования к хранению и обработке данных:

- **Надежность:** Все данные должны храниться в файловой системе. Потеря данных недопустима.
- **Ограничения целостности (Constraints):** Таблицы должны поддерживать типизацию столбцов и следующие модификаторы:
 - NOT_NULL — запрет на хранение пустых значений (по умолчанию поля nullable).
 - INDEXED — поле уникально, не может быть NULL. Для таких полей должен автоматически создаваться индекс на основе структуры данных согласно выданному варианту.
- **Оптимизация:** При выполнении запросов выборки (condition) необходимо использовать имеющиеся индексы для оптимизации поиска.
- **Хранение индексов:** Дублирование самих данных записей для построения индексов запрещено (индекс должен ссылаться на данные).
- **Валидация:** Все запросы должны проходить синтаксическую и семантическую валидацию. В случае ошибки система должна возвращать информативное описание проблемы.
- **Формат вывода:** Результатом успешного запроса выборки в терминал является массив объектов в формате JSON.

Программа не должна завершаться аварийно.

Синтаксис языка запросов:

Команды могут занимать несколько строк и завершаются символом ;. Ключевые слова регистронезависимы (смешение регистров в одном слове не допускается). Строковые литералы заключаются в двойные кавычки ("). Имена сущностей (БД, таблиц, колонок) могут содержать латинские буквы, цифры и подчеркивания, но не могут начинаться с цифры. Обращение к таблицам возможно в формате `database_name.table_name` либо через предварительный выбор контекста (USE).

Список поддерживаемых операций:

1. Работа с метаданными СУБД:

- `CREATE DATABASE [database_name];` — создание базы данных.
- `DROP DATABASE [database_name];` — удаление базы данных.
- `USE [database_name];` — выбор активной базы данных.

2. Работа со схемами данных (DDL):

- `CREATE TABLE [table_name] (col_name_1 col_type_1 <NOT_NULL|INDEXED>, ...);` — создание таблицы.
- `DROP TABLE [table_name];` — удаление таблицы.

3. Манипулирование данными (DML):

- `INSERT INTO [table_name] (col_1, ...) VALUE (val_1, ...), ...;` — вставка данных. Пропущенные столбцы без NOT_NULL инициализируются NULL.
- `UPDATE [table_name] SET col_1 = val_1, ... WHERE condition;` — обновление записей.
- `DELETE FROM [table_name] WHERE condition;` — удаление записей.
- `SELECT *|([col_1] <AS [alias]>, ...) FROM [table_name] <WHERE condition>;` — выборка данных с возможностью задания алиасов.

Условия выборки (condition):

Допускаются логические операции сравнения над именами столбцов и константами: `==`, `!=`, `<`, `>`, `<=`, `>=`.

При работе со строками сравнение необходимо осуществлять лексикографическое сравнение.

Для диапазонов: `val_1 BETWEEN val_2 AND val_3` (интервал `[val_2, val_3]`).

Для строк (Regex): `val_1 LIKE val_2`, где `val_2` — регулярное выражение.

Допустимо использование как переменных, так и констант в качестве любого значения.

Дополнительные задания:

Перед описанием задания в квадратных скобках указываются номера заданий, которые необходимо выполнить для выполнения текущего.

1. **[0]** Реализуйте механизм темпоральной персистентности, позволяющий откатывать состояние всей базы данных к определенному моменту времени. Команда: `REVERT [table_name] [yyyy.mm.dd-hh:mm:ss.msmsms];`. Реализация через полное копирование (snapshot) файлов БД запрещена.

2. **[0]** Обеспечьте оптимизацию хранения строковых данных в оперативной памяти (String Interning/Deduplication). Уникальные строки должны храниться в единственном экземпляре, а все использования в таблицах должны быть заменены ссылками.
3. **[0]** Реализуйте архитектуру "Клиент-Сервер". Основной функционал СУБД должен быть вынесен в серверное приложение. Клиентское приложение (терминал) должно отправлять запросы и получать ответы, используя средства межпроцессного взаимодействия (IPC) или сетевые сокеты.
4. **[0, 3]** Реализуйте кластерную архитектуру серверной части. Кластер должен состоять из многопоточного узла-балансировщика (Entrypoint) и произвольного количества узлов хранения (Storage). Entrypoint принимает запросы и делегирует их Storage-серверам, обеспечивая равномерное распределение данных (шардирование). Поддержите динамическое добавление/удаление узлов хранения без остановки системы.
5. **[0, 3]** Реализуйте механизм проверки доступности узлов (Heartbeat). Entrypoint должен периодически опрашивать Storage-серверы; при отсутствии ответа целевой сервер должен быть принудительно перезапущен.
6. **[0, 3]** Реализуйте асинхронную модель обработки запросов на основе очереди запросов. При отправке длительной операции сервер сразу возвращает идентификатор запроса (GUID v4). Должен быть реализован механизм (API) для последующего получения статуса и результата выполнения по этому идентификатору.
7. **[0]** Реализуйте подсистему логирования активности (Access Logs). Серверное приложение должно записывать в файл информацию о каждом запросе: тело запроса, ID клиента, ID обработчика, время начала и завершения обработки, статус/код возврата.
8. **[0]** Реализуйте децентрализованную подсистему сбора телеметрии. Система должна вычислять и отображать в реальном времени метрики производительности: текущий RPS (запросов в секунду), средний и максимальный RPS за 10 минут, среднее время обработки за 10 секунд, количество ошибок (Error Rate) за последнюю минуту.
9. **[0, 3]** Реализуйте подсистему разграничения доступа (RBAC) и аутентификации.
 - Для каждой базы данных настраиваются права по умолчанию для пользователей и групп. Необходимо разграничить доступ к чтению, записи данных в таблицу, созданию, удалению таблиц, удалению базы данных. Пользователь имеет право на выполнение операции, если это разрешают доступа по умолчанию, он состоит хотя бы в одной группе, имеющей доступ, или если доступ выделен лично для него.
 - Реализуйте хранилище аккаунтов. Пароли запрещено хранить в открытом виде (использовать хэширование с солью).
 - Аутентификация должна производиться с выдачей JWT-токена, который в дальнейшем используется для авторизации запросов.
10. **[0]** Реализуйте поддержку значений по умолчанию. Расширьте синтаксис CREATE TABLE модификатором DEFAULT [value]. При вставке данных (INSERT), если значение для столбца не передано, должно использоваться значение по

умолчанию.

11. **[0]** Реализуйте поддержку составных логических выражений в условных конструкциях (WHERE). Расширьте синтаксис запросов операторами булевой алгебры AND и OR для объединения предикатов. Обеспечьте корректный парсинг и вычисление приоритета операций, включая поддержку группировки условий с помощью круглых скобок (...).
12. **[0]** Реализуйте поддержку агрегатных функций SUM, COUNT, AVG в команде SELECT, которые должны осуществлять расчет суммы, количества строк, среднего среди всех подходящих значений выбранного столбца соответственно.

Для получения положительной (3 и выше) оценки за курсовую работу необходимо подготовить и сдать на кафедру пояснительную записку. Требования к оформлению пояснительной записки указаны ниже.

В основной части пояснительной записки необходимо для каждого реализованного задания описать алгоритм решения задания, использованные средства языка программирования C++, а также использованные при разработке алгоритмы и структуры данных.

Структура пояснительной записки:

- Титульный лист
- Содержание
- Введение
- Основная часть
- Вывод
- Список использованных источников
- Приложения

Оформление пояснительной записки:

- Поля: левое 20мм, остальные 15мм
- Нумерация страниц: начиная с титульного листа, индексация инкрементальная начиная с 1, по центру страницы; на титульном листе номер страницы не указывается
- Заголовки и подзаголовки разделов: шрифт Times New Roman 16pt, междустрочный интервал 1.5pt, выравнивание по левому краю
- Основной текст: шрифт Times New Roman 14pt, междустрочный интервал 1.15pt, выравнивание по ширине; абзацные отступы
- Рисунки: выравнивание по центру; под рисунком должна находиться подпись в формате
Рисунок #. <Описание рисунка>, где # - номер рисунка при сквозной нумерации рисунков по всей пояснительной записке, индексация инкрементальная начиная с 1. Оформление подписи к рисунку: шрифт Times New Roman 12pt, курсивный, междустрочный интервал 1pt, выравнивание по центру; рисунок и подпись к нему должны находиться на одной странице
- Таблицы: Выравнивание по центру; над таблицей должна находиться подпись в формате
Таблица #. <Описание таблицы>, где # - номер таблицы при сквозной нумерации таблиц по всей пояснительной записке, индексация инкрементальная начиная с 1. Оформление подписи к таблице: шрифт Times New Roman 12pt, курсивный, междустрочный интервал 1pt, выравнивание по левому краю; таблица и подпись к ней должны находиться на одной странице

- Листинги: Шрифт Consolas 12pt, междустрочный интервал 1pt, выравнивание по левому краю; над листингом должна находиться подпись в формате Листинг #. <Описание листинга>, где # - номер листинга при сквозной нумерации листингов по всей пояснительной записке, индексация инкрементальная начиная с 1. Оформление подписи к листингу: шрифт Times New Roman 12pt, курсивный, междустрочный интервал 1pt, выравнивание по левому краю; листинг и подпись к нему должны находиться на одной странице
- Список использованных источников: оформление по ГОСТ 7.0.100-2018

Во время защиты курсовой работы необходимо уметь ориентироваться в коде, демонстрировать работу реализованного комплекса приложений, быть готовым отвечать на вопросы по языку программирования C++ и по алгоритмам и структурам данных.

Оценивается только **финальная** версия реализованного комплекса приложений.

Защита курсового проекта без распечатанной пояснительной записки не проводится